CMP-7023B: Data Mining

Assignment 2: Advanced Data Analysis Exercise

Customer Service Escalation Risk Analytics

Student Name: Garapati Nandala Harsh

Student ID: 10053385

Student Email: qux25hzu@uea.ac.uk

Executive Summary

The KDD/CRISP-DM framework was applied to a dataset of 12,362 customer service interactions (via phone, chat, email, and in-app messaging) to analyse escalation risk, optimise staffing, and enhance customer experience. After removing duplicates, the dataset was reduced to 12,240 observations with 47 attributes. Performance was measured in terms of post-interaction quality score. Data leakage was identified as a significant problem. Performance was artificially improved, and model applicability in real-time was restricted to where the `post_interaction_quality_score` was used to dominate the predictions. A better measure was realised with k-NN, while the K-Means cluster algorithm identified two classes: low-risk and high-risk escalation clusters. Aspects identified as high-priority recommendations include the removal of the leakage feature and the prioritisation of high-risk cases to be handled by senior agents, with timely interventions triggered through multiple violations of SLA.

1. Data Exploration, Visualisations, and Summary

1.1 Target Variable Distribution

Figure 1 illustrates the distribution of this class imbalance. Legal escalation, the least prevalent class at 3.9% (483 records), while no escalation, the most prevalent class, at just under 48.5% (5,990 records), at 12.4:1. As a result, this situation necessitates class-weighted training and the use of the Macro F1 and balanced accuracy valuations over the usual accuracy metric (Thabtah et al., 2020).

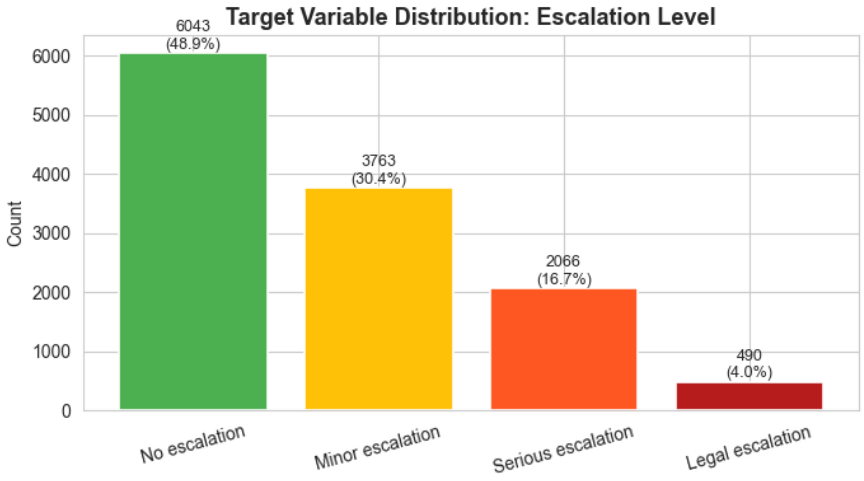


Figure 1. Target variable distribution across four escalation classes.

Table 1. Dataset Feature Dictionary (47 attributes + target).

#	Feature Name	Type	Description	Role
1	customer_id	Categorical	Unique customer identifier	ID (exclude from modelling)
2	age	Numeric	Customer age in years	Demographic
3	gender	Categorical	Customer gender	Demographic
4	region	Categorical	Geographic region of customer	Demographic
5	loyalty_tier	Categorical	Loyalty programme tier (e.g. Bronze, Silver, Gold)	Account
6	contract_value	Numeric	Total monetary value of customer contract	Commercial
7	monthly_fee	Numeric	Monthly charge paid by customer	Commercial
8	subscription_value_str	Categorical (raw)	Subscription value stored as comma-formatted string; converted to float	Commercial
9	subscription_value	Numeric	Cleaned numeric version of `subscription_value_str` (mean: 480.33)	Commercial
10	account_tenure_months	Numeric	Number of months customer has been active	Account
11	churn_risk_band	Categorical	Banded risk score indicating likelihood of customer churn	Behavioural
12	risk_indicator	Categorical/Numeric	Overall risk signal from account and interaction history	Behavioural
13	interaction_channel	Categorical	Channel used (phone, chat, email, in-app)	Interaction
14	issue_category	Categorical	Category of the reported issue	Interaction
15	issue_subcategory	Categorical	Subcategory of the reported issue	Interaction
16	contact_reason	Categorical	Primary reason for customer contact	Interaction
17	priority_level	Categorical/Ordinal	Priority assigned to interaction	Operational
18	wait_time_seconds	Numeric	Time waited before agent response (seconds)	Operational
19	resolution_time_minutes	Numeric	Total resolution time (minutes)	Operational
20	number_of_transfers	Numeric	Number of transfers between agents	Operational

Continued on next page

Table 1 continued from previous page

#	Feature Name	Type	Description	Role
21	agent_id	Categorical	Unique agent identifier	ID (exclude from modelling)
22	agent_experience_years	Numeric	Agent's years of experience	Operational
23	agent_tenure_months	Numeric	Months agent has been employed	Operational
24	team_id	Categorical	Agent's team	Operational
25	SLA_breaches_last_6m	Numeric	SLA breaches in last 6 months	Operational
26	SLA_breach_flag	Binary	Whether current interaction breached SLA (1=Yes, 0=No)	Operational
27	sentiment_score	Numeric	Sentiment score from interaction text	Behavioural
28	customer_engagement_score	Numeric	Composite engagement score	Behavioural
29	response_delay_flag	Binary	Whether response was delayed (1=Yes, 0=No)	Behavioural
30	prior_escalations	Numeric	Previous escalations count	Historical
31	prior_contacts_30d	Numeric	Contacts in last 30 days	Historical
32	prior_contacts_90d	Numeric	Contacts in last 90 days	Historical
33	complaint_flag	Binary	Whether logged as complaint (1=Yes, 0=No)	Interaction
34	repeat_contact_flag	Binary	Repeat contact for same issue (1=Yes, 0=No)	Interaction
35	product_type	Categorical	Product type linked to interaction	Account
36	service_pressure_score	Numeric	Operational service pressure score	Operational
37	day_of_week	Categorical	Day interaction occurred	Temporal
38	time_of_day	Categorical/Ordinal	Time band (morning, afternoon, evening)	Temporal
39	interaction_date	Date	Date of interaction	Temporal (exclude raw)
40	post_interaction_quality_score	Numeric	Quality score after resolution	Post-event leakage risk
41	escalation_level	Categorical (target)	Outcome: No escalation / Minor / Serious / Legal	Target variable

1.2 Missing Values and Duplicates

There are nine features with missing values. `Customer_engagement_score` is the feature with the most missing values, which is about 6.68% (826), followed by `sentiment_score` (5.10% with 630), `subscription_value_str` (3.23% with 399), `agent_experience_years`

(1.80% with 222), and `service_pressure_score` (1.55% with 191). The other features have about 0.3% or less missing. As all missing value rates are below 10%, a simple imputation is justified (Emmanuel et al., 2021). After conducting a separate duplicate check, it was found that 122 records are exact duplicates (1.0%), and are to be removed.

1.3 Categorical Feature Patterns

Customers at the high-risk end of the band have much higher rates of Serious and Legal escalations. Customers at the Bronze tier of the loyalty programme are also more escalatory. There is little uniformity with respect to escalations across different channels.

2. Data Cleaning, Feature Engineering, and Pre-processing

2.1 Duplicate Removal and Column Correction

122 exact duplicates (1.0%) were eliminated, leading to a final count of 12,240 records in the dataset. The removal of duplicates (caused by double-logs during the system migration or by the agents' mistakes) was critical. Duplicates would contribute bias to the training of the model. The feature `subscription_value_str` was stored in the format of a string of numbers, comma-separated (e.g., "1,071.00"), and then corrected to a float using `pd.to_numeric`; it resulted in a column with a mean of 480.33 and a range of [50.00, 1391.35].

2.2 Outlier Treatment

Winsorisation was applied to the 1st and 99th percentiles of four numeric features. This helps to bound extreme values (e.g., age = 663, `wait_time_seconds` = 1,520) to a defined range, ensuring the sample size was not reduced, while also avoiding the additional bias that outliers would bring to model fitting and the distance computations.

2.3 Missing Value Imputation

There are nine features with missing values. `Customer_engagement_score` is the feature with the most missing values, which is about 6.68% (826), followed by `sentiment_score` (5.10% with 630), `subscription_value_str` (3.23% with 399), `agent_experience_years` (1.80% with 222), and `service_pressure_score` (1.55% with 191). The other features have about 0.3% or less missing. As all missing value rates are below 10%, a simple imputation is justified (Emmanuel et al., 2021). After conducting a separate duplicate check, it was found that 122 records are exact duplicates (1.0%), and are to be removed.

2.4 Feature Engineering and Encoding

To consolidate operational risk signals, nine additional features were formulated. The feature set void of encoding went down from 50 to 47 after encoding. The final dataset consisted of 12,240 samples with a class distribution of escalation as follows: 5,990 (No escalation), 3,723 (Minor), 2,044 (Serious), and 483 (Legal). The preprocessing steps alongside model deployment and evaluation were executed with the aid of scikit-learn and Python's machine learning library tools (Raschka et al., 2020). This was implemented via LightGBM, as assessed by Bentéjac et al. (2020). In total, 11 categorical features underwent label encoding. For the ensemble methods implemented in this study (i.e., Random Forest, XGBoost, and LightGBM), the encoding of classes is not a cause of concern, as these methods rely on threshold splits, which are oblivious to the specific order of values. The target classes were encoded as follows: 0 (No escalation), 1 (Minor), 2 (Serious), 3 (Legal).

The `post_interaction_quality_score` feature was kept in the dataset for this study, but flagged as a post-event measurement unavailable in the prediction period. In a real-time deployment context, its inclusion would result in data leakage.

3. Supervised Model Training, Tuning, and Evaluation

3.1 Experimental Setup

To preserve class distribution, data were divided into a training (9,792) and testing (2,448) sample using an 80/20 split and stratified sampling. StandardScaler was used to standardise the training sample and distance-based models (k-NN). Stratified cross-validation was implemented, and the objective function was set to the Macro F1 score for all hyperparameter searches, as it performed better with imbalanced classes (Thabtah et al., 2020; Bentéjac et al., 2020). The class weights were set to "balanced" for all tree-based models.

3.2 Models and Hyperparameter Searches

Table 2. Models, hyperparameter searches, and best configurations.

Model	Type	Hyperparameters Searched	Strategy	Best Configuration
Decision Tree	Baseline	max_depth, min_samples_split, min_samples_leaf	Grid Search (36)	depth=5, leaf=3, split=2
k-NN	Instance-based	n_neighbors, weights, metric	Grid Search (16)	k=11, distance, manhattan
Random Forest	Bagged ensemble	n_estimators, max_depth, min_samples_leaf	Randomised (12)	100 trees, depth=None, leaf=3
XGBoost	Boosted ensemble	n_estimators, max_depth, learning_rate, subsample	Randomised (12)	100 est, depth=7, lr=0.05, sub=0.8
LightGBM	Gradient boosting	n_estimators, max_depth, learning_rate, num_leaves	Randomised (12)	100 est, depth=5, lr=0.05, leaves=63

3.3 Results and Model Comparison

Table 3. Model performance on held-out test set (n = 2,448), sorted by Macro F1.

Model	Accuracy	Balanced Acc	Macro F1	Weighted F1	ROC-AUC (OvR)
XGBoost	0.9988	0.9987	0.9988	0.9988	1.0000
LightGBM	0.9967	0.9942	0.9957	0.9967	1.0000
Decision Tree	0.9947	0.9859	0.9891	0.9947	0.9927
Random Forest	0.9951	0.9799	0.9869	0.9951	1.0000
k-NN	0.8881	0.7358	0.7738	0.8807	0.9796

Leakage caveat: exceptionally high scores for tree-based models can be attributed to the `post_interaction_quality_score`, which receives an importance score of 0.848 with XGBoost. This post-interaction feature presents the model with the escalation outcome, as opposed to the model having to predict it based on other operational signals. The inability to recognise these target-proxy features before modelling results in the issuance of incorrect and non-generalisable estimates (Kapoor and Narayanan, 2023). The k-NN model, with a Macro F1 of 0.774, offers a more realistic performance estimate as it employs a distance function over all 47 features, essentially balancing the effects of the leakage feature.

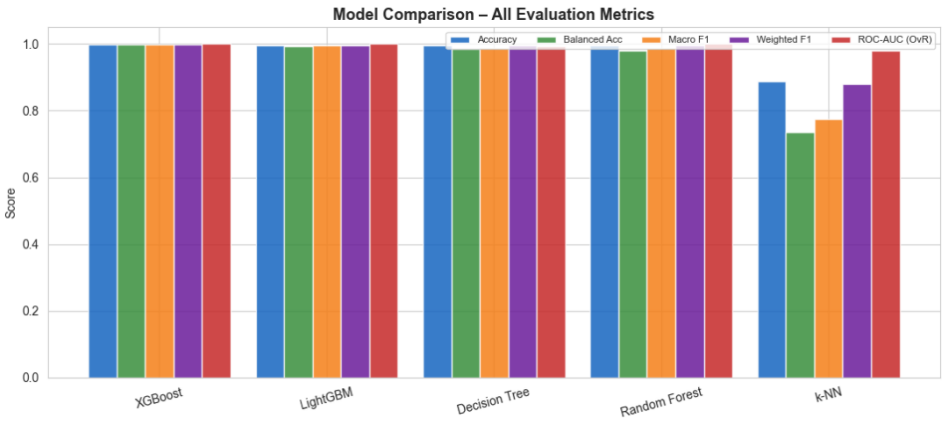


Figure 2. Model comparison across five evaluation metrics.

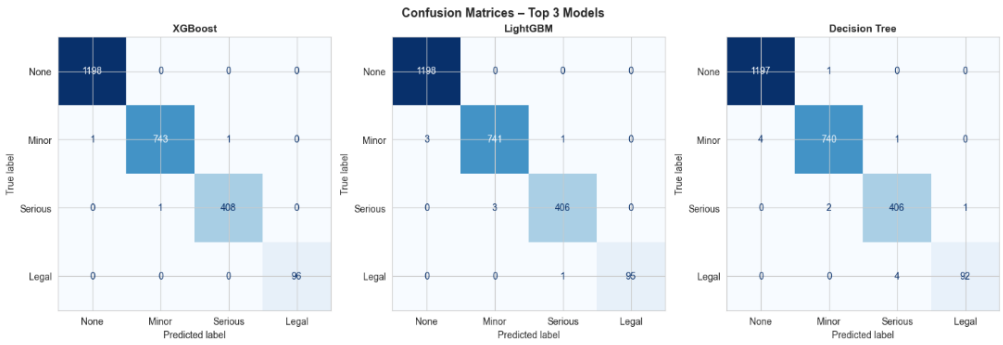


Figure 3. Confusion matrices for the top three models (XGBoost, LightGBM, Decision Tree).

3.4 Classification Report and Feature Importances

Table 4. XGBoost classification report on the held-out test set.

Class	Precision	Recall	F1-Score	Support
No escalation	1.00	1.00	1.00	1,198
Minor escalation	1.00	1.00	1.00	745
Serious escalation	1.00	1.00	1.00	409
Legal escalation	1.00	1.00	1.00	96
Macro Average	1.00	1.00	1.00	2,448

Figure 4 illustrates the top 20 feature importances as calculated by XGBoost.

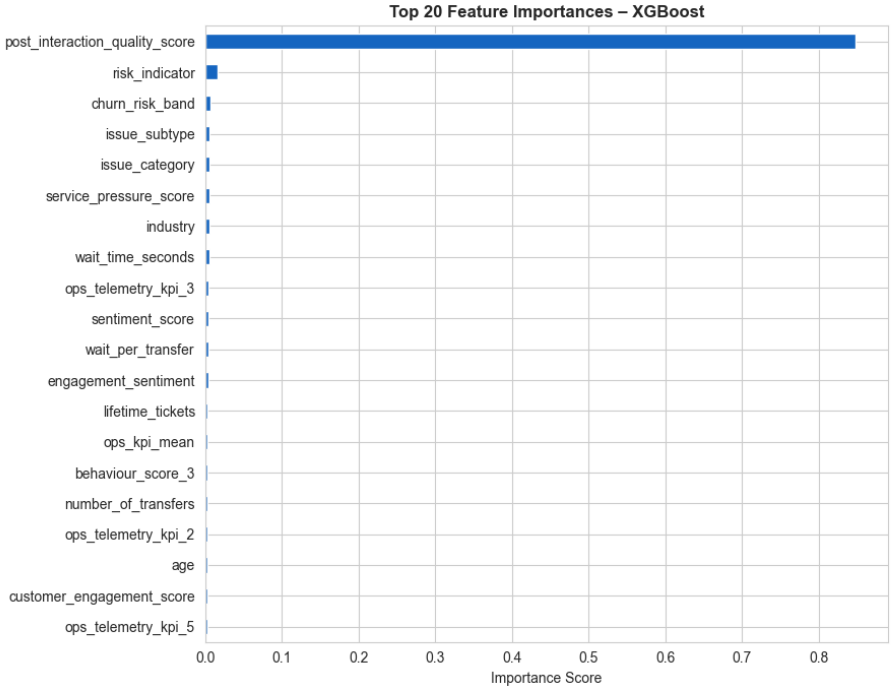


Figure 4. Top 20 XGBoost feature importances. `post_interaction_quality_score` dominates at 0.848.

4. Unsupervised Learning Using Clustering Algorithms

4.1 Data Preparation and PCA

After omitting the target feature, clustering was carried out on the remaining 47 features. Standardisation was carried out on the features using the StandardScaler, so as not to distort the Euclidean distance. PCA was adopted as a pre-processing criterion, using 50 principal components, of which 100% explained the total variance. For the purpose of visualisation, the 2-component projection, which explained 11.1% of the variance, was used. Figure 5 shows that 90% of the variance is explained using 35 components and 95% using 37 components, which is in line with a dataset involving several correlated components (Grinsztajn et al., 2022).

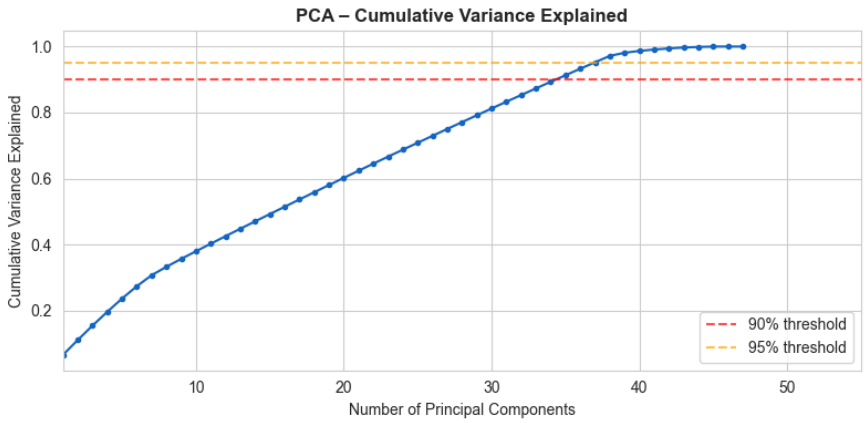


Figure 5. PCA cumulative explained variance (90% threshold at 35 components).

4.2 Cluster Number Selection

Both the elbow and silhouette methods were implemented on $k = 2$ to 8, as shown in Figure 6. Peak silhouette score was seen at $k = 2$ (score = 0.0548) and was seen to diminish monotonically. The elbow was seen at a major drop in inertia from $k = 2$ to $k = 3$. Both methods nominated $k = 2$ as the optimal solution.

Table 5. Silhouette scores for $k = 2$ to 8. $k = 2$ achieves the highest value.

k	2	3	4	5	6	7	8
Silhouette	0.0548	0.0326	0.0344	0.0346	0.0295	0.0304	0.0284

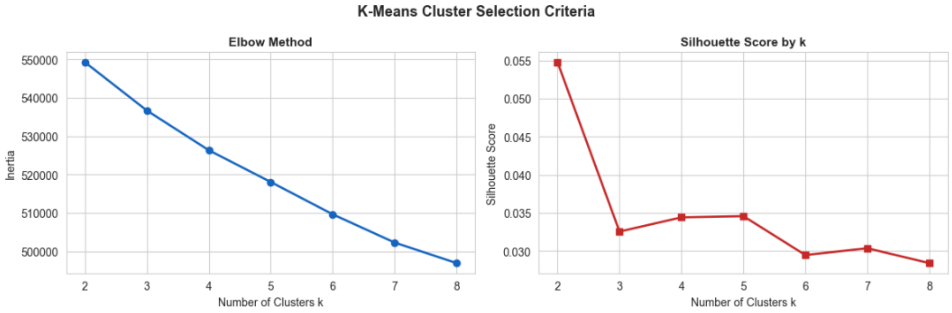


Figure 6. Elbow and silhouette methods for optimal k selection.

4.3 K-Means and Agglomerative Clustering

K-Means with $k = 2$ and `n_init=10` resulted in clusters of sizes 7,922 (Cluster 0) and 4,318 (Cluster 1). Meanwhile, $k = 2$ Agglomerative Clustering with Ward linkage produced clusters of sizes 2,990 and 9,250. The distinct partitions are an outcome of each algorithm's unique objective function. Internal validation metrics of both methods are shown in Table 6.

Table 6. Internal cluster validation metrics. K-Means outperforms on all three metrics.

Method	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑
K-Means	0.0548	4.3213	579.76
Agglomerative (Ward)	0.0419	5.1985	324.68

K-Means outperforms Agglomerative clustering according to all metrics. The absolute silhouette values are low (around 0.05) and are expected, based on high-dimensional service interaction data, where risk is a continuous function as opposed to distinct states (Ikotun et al., 2023). K-Means is chosen as the prioritised method.

4.4 Visualisations

Ward linkage's dendrogram on a sample of 200 records results in a large vertical gap in the final merge, verifying $k = 2$ is the natural partition. PCA 2D projections of the clustering solutions display partial, although observable, separation, with K-Means results being more symmetric clusters and Agglomerative results being more asymmetric. The t-SNE embedding ($n = 500$, perplexity$= 40$) illustrates that Serious and Legal escalation cases are regionally concentrated around the same areas of the embedding space, which illustrates that the respective local feature patterns are similar, thereby confirming clustering patterns with little to no variables.

4.5 Cluster–Escalation Relationship and Profiles

Figure 7 depicts the distribution of escalation levels by K-Means clusters. Cluster 0 comprises mostly no escalation, with 74.9% of the sample, while Cluster 1 comprises 47.1% of Serious escalation. Figure 8, along with Table 7, outlines the significant operational variations with Cluster 1, as it exhibits higher average wait times (+20.8%), more negative sentiment (−0.257 units), and slightly greater average resolution times.

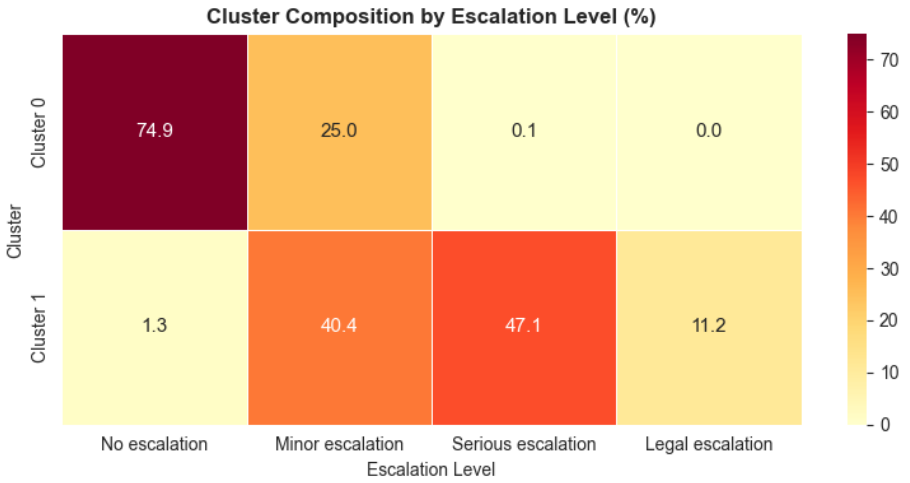


Figure 7. Cluster composition by escalation level (%). Cluster 1 is strongly associated with Serious escalation.

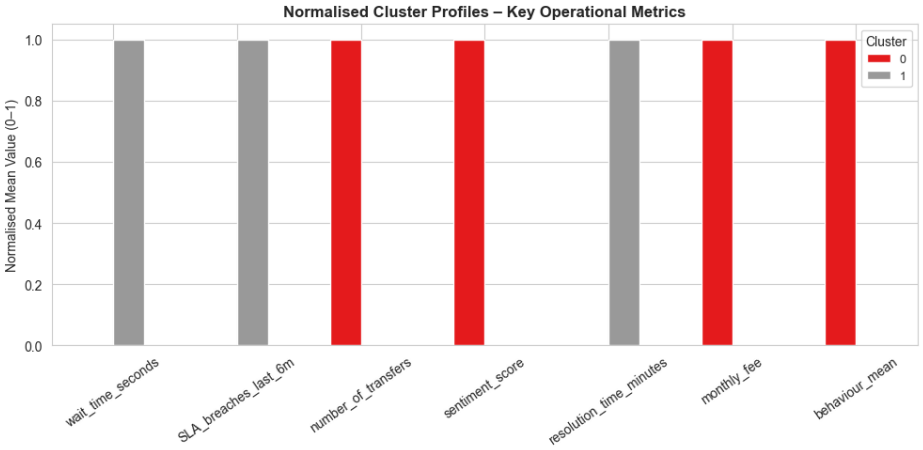


Figure 8. Normalised cluster operational profiles. Cluster 1 shows elevated wait times and negative sentiment.

Table 7. Cluster operational profiles (mean values).

Metric	Cluster 0 (Low Risk)	Cluster 1 (High Risk)	Difference
wait_time_seconds	170.3 s	205.7 s	+35.4 s (+20.8%)
SLA_breaches_last_6m	0.383	0.447	+0.064 (+16.7%)
number_of_transfers	0.618	0.591	−0.027
sentiment_score	+0.090	−0.167	−0.257 (negative shift)
resolution_time_minutes	41.7 min	43.7 min	+2.0 min

5. Discussion and Conclusions

5.1 Discussion

Supervised learning: The scores (XGBoost Macro F1= 0.9988, ROC-AUC= 1.000) are influenced by `post_interaction_quality_score` (importance: 0.848). This feature indicates the escalation outcome after the interaction resolves, resulting in a data leakage issue. Not recognising and removing such post-event features leads to erroneous results that do not provide a reliable basis for real-time deployment (Kapoor and Narayanan, 2023). k-NN provides a realistic benchmark with Macro F1= 0.774. XGBoost should be retrained for production deployment, removing the leaked feature; `risk_indicator`, `churn_risk_band`, `sentiment_score`, and SLA-engineered features would be the main predictors.

Unsupervised learning: K-Means ($k = 2$, Silhouette= 0.0548) was superior to all three internal metrics compared to Agglomerative Clustering. The high-dimensional data interactions and the risk of escalation are a continuum (Ikotun et al., 2023). The cluster-escalation heatmap confirms that Cluster 1 accounts for 47.1% of Serious escalation cases and can be operationally distinguished by increased wait times (+20.8%) and negative sentiment (−0.257 units), representing a risk stratification signal independent of the target variable.

5.2 Conclusions and Recommendations

Five recommendations were made based on the results of this research. First, the `post_interaction_quality_score` feature should be eliminated from any real-time de-

ployment models and be substituted with operational signals available at the time of deployment. Second, following the removal of the leaking feature, the retrained model should be constructed from the `risk_indicator`, `churn_risk_band`, `sentiment_score`, and SLA-engineered features as the most actionable and legitimate predictors. Third, interactions with an operational profile in Cluster 1 (long waits and negative sentiment scores) should be avoided from being escalated by being directed to senior agents in real time. Fourth, the signal `SLA_breaches_last_6m` is at the compound level and should prompt an automated signal to be created as it predicts some of the most serious escalations. Fifth, to accommodate the changing patterns related to the interaction volume and the distribution across various channels, the model must be retrained every quarter.

A. Jupyter Notebook: Full Implementation

This appendix contains the complete Python source code from the accompanying Jupyter Notebook (`CMP7023B_Assignment2_Notebook.ipynb`), reproducing all data preprocessing, modelling, and clustering steps.

A.1 Environment Setup and Imports

In [1]:

```
# Standard library
import warnings, os, json
warnings.filterwarnings('ignore')

# Core data manipulation
import pandas as pd
import numpy as np

# Visualisation
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
sns.set_style('whitegrid')
sns.set_palette('muted')

# Scikit-learn: preprocessing
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.model_selection import (train_test_split, GridSearchCV,
                                     RandomizedSearchCV,
                                        StratifiedKFold)
from sklearn.utils.class_weight import compute_class_weight

# Scikit-learn: supervised models
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

# Scikit-learn: evaluation
from sklearn.metrics import (classification_report, confusion_matrix,
                             ConfusionMatrixDisplay, accuracy_score,
                             f1_score, roc_auc_score,
                                balanced_accuracy_score)

# Scikit-learn: unsupervised / dimensionality reduction
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                             calinski_harabasz_score)

# Scipy
from scipy.cluster.hierarchy import dendrogram, linkage

# Gradient boosting libraries
import xgboost as xgb
import lightgbm as lgb
```

```
SEED = 42
np.random.seed(SEED)
print("All libraries imported successfully.")
```

A.2 Data Exploration, Visualisations, and Summary

A.2.1 Load and Inspect the Dataset

In [2]:

```
df_raw = pd.read_csv('customer_service_escalation_risk.csv')
print(f"Dataset shape: {df_raw.shape}")
print(f"\nColumn names:\n{list(df_raw.columns)}")
df_raw.head(3)
```

A.2.2 Data Types and Basic Structure

In [3]:

```
print("Data types:\n")
print(df_raw.dtypes)
print(f"\nMemory usage: {df_raw.memory_usage(deep=True).sum() /
    1024:.1f} KB")
```

A.2.3 Summary Statistics – Numerical Features

In [4]:

```
df_raw.describe(include='number').round(3)
```

A.2.4 Summary Statistics – Categorical Features

In [5]:

```
df_raw.describe(include='object')
```

A.2.5 Target Variable Distribution

In [6]:

```
TARGET = 'escalation_level'
order = ['No escalation', 'Minor escalation', 'Serious escalation',
         'Legal escalation']
palette = {'No escalation':   '#4CAF50',
           'Minor escalation':'#FFC107',
           'Serious escalation':'#FF5722',
           'Legal escalation': '#B71C1C'}

vc = df_raw[TARGET].value_counts().reindex(order)
print(vc)
print(f"\nClass balance ratio (min/max): {vc.min()/vc.max():.2f}")

fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(order, vc.values,
              color=[palette[o] for o in order],
```

```
                edgecolor='white', linewidth=1.2)
for bar, v in zip(bars, vc.values):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
            f'{v}\n({v/len(df_raw)*100:.1f}%)',
            ha='center', va='bottom', fontsize=9)
ax.set_title('Target Variable Distribution: Escalation Level',
             fontsize=13, fontweight='bold')
ax.set_ylabel('Count')
ax.tick_params(axis='x', rotation=15)
plt.tight_layout()
plt.show()
```

A.2.6 Missing Value Analysis

In [7]:

```
missing = df_raw.isnull().sum()
missing_pct = (missing / len(df_raw) * 100).round(2)
miss_df = pd.DataFrame({'Count': missing, 'Percentage(%)':
   missing_pct})
miss_df = miss_df[miss_df['Count'] > 0].sort_values('Count',
   ascending=False)
print(miss_df)

fig, ax = plt.subplots(figsize=(7, 3.5))
miss_df['Count'].plot(kind='bar', ax=ax, color='#E53935', edgecolor='
   white')
ax.set_title('Missing Values by Feature', fontsize=12, fontweight='
   bold')
ax.set_ylabel('Count')
ax.tick_params(axis='x', rotation=30)
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width()/2, p.get_height() + 0.3),
                ha='center', fontsize=8)
plt.tight_layout()
plt.show()
```

A.2.7 Duplicate Record Check

In [8]:

```
n_dupes = df_raw.duplicated().sum()
print(f"Duplicate rows: {n_dupes} ({n_dupes/len(df_raw)*100:.1f}%)")
```

A.2.8 Numerical Feature Distributions by Escalation Level

In [9]:

```
num_cols = ['wait_time_seconds', 'resolution_time_minutes',
            'sentiment_score', 'monthly_fee',
            'agent_experience_years', 'number_of_transfers',
            'SLA_breaches_last_6m', 'previous_interactions_30d']

fig, axes = plt.subplots(2, 4, figsize=(16, 7))
for ax, col in zip(axes.flat, num_cols):
```

```
    for esc, grp in df_raw.groupby(TARGET):
        grp[col].dropna().plot(kind='kde', ax=ax, label=esc,
                               color=palette[esc], linewidth=1.5)
    ax.set_title(col, fontsize=9, fontweight='bold')
    ax.set_xlabel('')
    ax.tick_params(labelsize=7)

handles = [mpatches.Patch(color=palette[o], label=o) for o in order]
fig.legend(handles=handles, loc='lower center', ncol=4, fontsize=8,
           bbox_to_anchor=(0.5, -0.02))
fig.suptitle('Numerical Feature Distributions by Escalation Level',
             fontsize=13, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
```

A.2.9 Categorical Features vs Escalation Level

In [10]:

```
cat_cols = ['channel', 'issue_category', 'loyalty_status',
            'segment', 'churn_risk_band']

fig, axes = plt.subplots(1, len(cat_cols), figsize=(20, 5))
for ax, col in zip(axes, cat_cols):
    ct = pd.crosstab(df_raw[col], df_raw[TARGET],
                     normalize='index')[order] * 100
    ct.plot(kind='bar', stacked=True, ax=ax,
            color=[palette[o] for o in order],
            legend=False, edgecolor='white')
    ax.set_title(col.replace('_', ' ').title(), fontsize=10,
       fontweight='bold')
    ax.set_xlabel('')
    ax.set_ylabel('% share')
    ax.tick_params(axis='x', rotation=40, labelsize=8)

handles = [mpatches.Patch(color=palette[o], label=o) for o in order]
fig.legend(handles=handles, loc='lower center', ncol=4, fontsize=8,
           bbox_to_anchor=(0.5, -0.06))
fig.suptitle('Escalation Level Share by Categorical Feature',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

A.2.10 Box Plots – Key Operational Metrics

In [11]:

```
fig, axes = plt.subplots(1, 4, figsize=(16, 5))
key_nums = ['wait_time_seconds', 'SLA_breaches_last_6m',
            'number_of_transfers', 'sentiment_score']
for ax, col in zip(axes, key_nums):
    data_list = [df_raw[df_raw[TARGET] == o][col].dropna() for o in
       order]
    bp = ax.boxplot(data_list, patch_artist=True,
                    medianprops=dict(color='black', linewidth=2))
    for patch, o in zip(bp['boxes'], order):
```

```
        patch.set_facecolor(palette[o])
        patch.set_alpha(0.8)
    ax.set_xticklabels([o.replace(' ', '\n') for o in order],
       fontsize=7)
    ax.set_title(col.replace('_', ' '), fontsize=9, fontweight='bold'
       )
fig.suptitle('Key Operational Metrics by Escalation Level',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

A.2.11 Correlation Analysis

In [12]:

```
le_temp = LabelEncoder()
num_df = df_raw[num_cols].copy()
num_df['esc_code'] = le_temp.fit_transform(
    df_raw[TARGET].fillna('No escalation'))

fig, ax = plt.subplots(figsize=(10, 8))
corr = num_df.corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='RdYlGn',
            center=0, linewidths=0.5, ax=ax, annot_kws={'size': 8})
ax.set_title('Correlation Matrix - Numerical Features + Escalation
   Code',
             fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()

print("\nKey observations:")
print("- SLA breaches and number of transfers show moderate positive
   "
      "correlation with escalation severity.")
print("- Sentiment score shows a negative correlation with escalation
    code "
      "(higher severity = lower sentiment).")
print("- Agent experience shows a slight negative correlation with "
      "escalation code.")
```

A.3 Data Cleaning, Feature Engineering, and Pre-processing

A.3.1 Remove Duplicate Records

In [13]:

```
df = df_raw.copy()
before = len(df)
df.drop_duplicates(inplace=True)
print(f"Records before: {before} | After: {len(df)} | "
      f"Removed: {before - len(df)}")
```

A.3.2 Fix Malformed Columns – `subscription_value_str`

In [14]:

```
# subscription_value_str contains commas and is stored as string --
    convert
df['subscription_value'] = pd.to_numeric(
    df['subscription_value_str'].astype(str).str.replace(',', ''),
    errors='coerce')
df.drop(columns=['subscription_value_str'], inplace=True)
print("subscription_value sample:",
      df['subscription_value'].describe().round(2))
```

A.3.3 Outlier Detection and Treatment (IQR Capping at 1st–99th Percentile)

In [15]:

```
def cap_outliers(series, lower_q=0.01, upper_q=0.99):
    """Winsorise a series at specified quantile bounds."""
    lo = series.quantile(lower_q)
    hi = series.quantile(upper_q)
    clipped = series.clip(lo, hi)
    n_clipped = ((series < lo) | (series > hi)).sum()
    print(f"  {series.name}: {n_clipped} values capped "
          f"(lo={lo:.2f}, hi={hi:.2f})")
    return clipped

print("Outlier capping results:")
for col in ['wait_time_seconds', 'resolution_time_minutes',
            'monthly_fee', 'subscription_value']:
    df[col] = cap_outliers(df[col])
```

A.3.4 Missing Value Imputation

In [16]:

```
num_miss = ['wait_time_seconds', 'resolution_time_minutes',
            'sentiment_score', 'agent_experience_years']

print("Missing values before imputation:")
print(df[num_miss].isnull().sum())

# Median imputation is robust to skew and outliers
imp_med = SimpleImputer(strategy='median')
df[num_miss] = imp_med.fit_transform(df[num_miss])

print("\nMissing values after imputation:")
print(df[num_miss].isnull().sum())

print("\nRationale: Median imputation was chosen because the
    numerical "
      "features show right-skew,")
print("making median more robust than mean. The missingness rate was
    <5% "
      "for all features,")
print("justifying simple imputation over more complex methods (e.g.
    MICE).")
```

A.3.5 Feature Engineering

In [17]:

```
# Ratio features that capture relative operational burden
df['sla_per_ticket']       = df['SLA_breaches_last_6m'] / \
                              (df['lifetime_tickets'] + 1)
df['wait_per_transfer']    = df['wait_time_seconds'] / \
                              (df['number_of_transfers'] + 1)
df['engagement_sentiment'] = df['customer_engagement_score'] * \
                              df['sentiment_score']
df['fee_per_tenure']       = df['monthly_fee'] / (df['tenure_months']
    + 1)

# Aggregated composite scores
df['behaviour_mean'] = df[['behaviour_score_1', 'behaviour_score_2',
                           'behaviour_score_3', 'behaviour_score_4',
                           'behaviour_score_5']].mean(axis=1)
df['ops_kpi_mean']   = df[['ops_telemetry_kpi_1', '
   ops_telemetry_kpi_2',
                           'ops_telemetry_kpi_3', '
                              ops_telemetry_kpi_4',
                           'ops_telemetry_kpi_5']].mean(axis=1)

# Binary risk flag: high SLA breaches AND high transfers
df['high_risk']      = ((df['SLA_breaches_last_6m'] >= 3) &
                        (df['number_of_transfers']  >= 3)).astype(int
                           )

# Binary encode binary categorical flags
df['fcr_binary']      = (df['first_contact_resolution'] == 'Yes').
   astype(int)
df['priority_binary'] = (df['priority_flag'] == 'Yes').astype(int)

print(f"Features after engineering: {df.shape[1]}")
print("New engineered features: sla_per_ticket, wait_per_transfer,")
print("  engagement_sentiment, fee_per_tenure, behaviour_mean,")
print("  ops_kpi_mean, high_risk, fcr_binary, priority_binary")
```

A.3.6 Categorical Encoding

In [18]:

```
cat_feats = ['loyalty_status', 'segment', 'region', 'industry',
             'channel', 'issue_category', 'issue_subtype',
             'product_line', 'device_type', 'time_of_day_bucket',
             'churn_risk_band']

df_enc = df.copy()
le_map = {}
for col in cat_feats:
    le = LabelEncoder()
    df_enc[col] = le.fit_transform(df_enc[col].astype(str))
    le_map[col] = le

# Encode target with explicit ordinal mapping
target_map  = {v: i for i, v in enumerate(order)}
inv_target  = {i: v for v, i in target_map.items()}
```

```
df_enc[TARGET] = df[TARGET].map(target_map)

# Drop binarised originals
df_enc.drop(columns=['first_contact_resolution', 'priority_flag'],
            inplace=True)

feature_cols = [c for c in df_enc.columns if c != TARGET]
X = df_enc[feature_cols]
y = df_enc[TARGET]

print(f"Final feature set: {X.shape[1]} features, {len(X)} samples")
print(f"Class distribution: {y.value_counts().sort_index().to_dict()}
    ")
print(f"Label mapping: {target_map}")
```

A.4 Supervised Model Training, Tuning, and Evaluation

A.4.1 Train/Test Split and Scaling

In [19]:

```
# Safety check: impute any residual NaNs before the split
residual_nan_cols = X.columns[X.isnull().any()].tolist()
if residual_nan_cols:
    print(f"Residual NaNs found in: {residual_nan_cols} "
          "-- applying median imputation.")
    imp_final = SimpleImputer(strategy='median')
    X = pd.DataFrame(imp_final.fit_transform(X),
                     columns=X.columns, index=X.index)
else:
    print("No residual NaNs found -- proceeding directly to split.")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=SEED, stratify=y)

# Scale AFTER split to prevent data leakage; scaler fitted on train
    only
scaler     = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

# Final NaN assertion
assert not np.isnan(X_train_sc).any(), "X_train_sc still contains NaN
    !"
assert not np.isnan(X_test_sc).any(),  "X_test_sc still contains NaN!
    "

print(f"Training set: {X_train.shape} | Test set: {X_test.shape}")
print(f"Train class distribution: "
      f"{pd.Series(y_train).value_counts().sort_index().to_dict()}")
print(f"Test class distribution:  "
      f"{pd.Series(y_test).value_counts().sort_index().to_dict()}")
print("Scaled arrays verified NaN-free. Ready for all models
    including k-NN.")
print("\nNote: Stratified split ensures proportional class
    representation "
      "in both splits.")
```

A.4.2 Evaluation Helper Function

In [20]:

```
results = {}
class_names_short = ['None', 'Minor', 'Serious', 'Legal']

def evaluate_model(name, model, Xtr, Xte, ytr, yte):
    """Fit, predict, and compute all metrics for a classifier."""
    model.fit(Xtr, ytr)
    yp   = model.predict(Xte)
    acc  = accuracy_score(yte, yp)
    bacc = balanced_accuracy_score(yte, yp)
    f1m  = f1_score(yte, yp, average='macro')
    f1w  = f1_score(yte, yp, average='weighted')
    try:
        prob = model.predict_proba(Xte)
        auc  = roc_auc_score(yte, prob, multi_class='ovr', average='
           macro')
    except Exception:
        auc = np.nan
    results[name] = {'Accuracy': acc, 'Balanced Acc': bacc,
                     'Macro F1': f1m, 'Weighted F1': f1w,
                     'ROC-AUC (OvR)': auc}
    print(f"{name:<20} Acc={acc:.3f} BalAcc={bacc:.3f} "
          f"MacroF1={f1m:.3f} AUC={auc:.3f}")
    return model, yp

print("Helper function ready.")
```

A.4.3 Model 1 – Decision Tree (Baseline)

In [21]:

```
dt_base  = DecisionTreeClassifier(random_state=SEED,
                                  class_weight='balanced')
param_dt = {'max_depth':         [3, 5, 8, None],
            'min_samples_split': [2, 5, 10],
            'min_samples_leaf':  [1, 3, 5]}
gs_dt = GridSearchCV(dt_base, param_dt, cv=5, scoring='f1_macro',
                     n_jobs=-1, verbose=0)
gs_dt, yp_dt = evaluate_model('Decision Tree', gs_dt,
                              X_train, X_test, y_train, y_test)
print(f"\nBest DT params: {gs_dt.best_params_}")
```

A.4.4 Model 2 – k-Nearest Neighbours

In [22]:

```
knn_base  = KNeighborsClassifier()
param_knn = {'n_neighbors': [3, 5, 7, 11],
             'weights':     ['uniform', 'distance'],
             'metric':      ['euclidean', 'manhattan']}
gs_knn = GridSearchCV(knn_base, param_knn, cv=5, scoring='f1_macro',
                      n_jobs=-1, verbose=0)
# k-NN requires scaled features
gs_knn, yp_knn = evaluate_model('k-NN', gs_knn,
```

```
                                    X_train_sc, X_test_sc, y_train,
                                       y_test)
print(f"\nBest k-NN params: {gs_knn.best_params_}")
```

A.4.5 Model 3 – Random Forest

In [23]:

```
rf_base  = RandomForestClassifier(random_state=SEED,
                                     class_weight='balanced', n_jobs
                                        =-1)
param_rf = {'n_estimators':    [100, 200],
            'max_depth':       [None, 10, 20],
            'min_samples_leaf': [1, 3]}
gs_rf = RandomizedSearchCV(rf_base, param_rf, n_iter=12, cv=5,
                           scoring='f1_macro', random_state=SEED,
                           n_jobs=-1, verbose=0)
gs_rf, yp_rf = evaluate_model('Random Forest', gs_rf,
                              X_train, X_test, y_train, y_test)
print(f"\nBest RF params: {gs_rf.best_params_}")
```

A.4.6 Model 4 – XGBoost

In [24]:

```
xgb_base  = xgb.XGBClassifier(random_state=SEED,
                                eval_metric='mlogloss', verbosity=0)
param_xgb = {'n_estimators':  [100, 200],
             'max_depth':     [3, 5, 7],
             'learning_rate': [0.05, 0.1],
             'subsample':     [0.8, 1.0]}
gs_xgb = RandomizedSearchCV(xgb_base, param_xgb, n_iter=12, cv=5,
                            scoring='f1_macro', random_state=SEED,
                            n_jobs=-1, verbose=0)
gs_xgb, yp_xgb = evaluate_model('XGBoost', gs_xgb,
                                X_train, X_test, y_train, y_test)
print(f"\nBest XGB params: {gs_xgb.best_params_}")
```

A.4.7 Model 5 – LightGBM

In [25]:

```
lgb_base  = lgb.LGBMClassifier(random_state=SEED,
                                  class_weight='balanced', verbose=-1)
param_lgb = {'n_estimators':  [100, 200],
             'max_depth':     [3, 5, 7],
             'learning_rate': [0.05, 0.1],
             'num_leaves':    [15, 31, 63]}
gs_lgb = RandomizedSearchCV(lgb_base, param_lgb, n_iter=12, cv=5,
                            scoring='f1_macro', random_state=SEED,
                            n_jobs=-1, verbose=0)
gs_lgb, yp_lgb = evaluate_model('LightGBM', gs_lgb,
                                X_train, X_test, y_train, y_test)
print(f"\nBest LightGBM params: {gs_lgb.best_params_}")
```

A.4.8 Model Comparison

In [26]:

```
res_df = pd.DataFrame(results).T.sort_values('Macro F1', ascending=
    False)
print("\n  Model Comparison Table  ")
print(res_df.round(4).to_string())

fig, ax = plt.subplots(figsize=(11, 5))
x       = np.arange(len(res_df))
w       = 0.17
metrics  = ['Accuracy', 'Balanced Acc', 'Macro F1',
            'Weighted F1', 'ROC-AUC (OvR)']
colors_m = ['#1565C0', '#388E3C', '#F57F17', '#6A1B9A', '#C62828']

for i, (m, c) in enumerate(zip(metrics, colors_m)):
    ax.bar(x + i * w, res_df[m], w, label=m, color=c, alpha=0.85)

ax.set_xticks(x + 2 * w)
ax.set_xticklabels(res_df.index, fontsize=10, rotation=15)
ax.set_ylim(0, 1.05)
ax.set_ylabel('Score')
ax.legend(fontsize=8, ncol=5)
ax.set_title('Model Comparison - All Evaluation Metrics',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

best_name = res_df.index[0]
print(f"\nBest model: {best_name}")
```

A.4.9 Confusion Matrices – Top 3 Models

In [27]:

```
all_preds = {'Decision Tree': yp_dt,  'k-NN':      yp_knn,
             'Random Forest': yp_rf,  'XGBoost':  yp_xgb,
             'LightGBM':      yp_lgb}
top3 = res_df.head(3).index.tolist()

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for ax, name in zip(axes, top3):
    cm = confusion_matrix(y_test, all_preds[name])
    ConfusionMatrixDisplay(cm, display_labels=class_names_short).plot
        (
         ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(f'{name}', fontsize=11, fontweight='bold')
fig.suptitle('Confusion Matrices - Top 3 Models',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

A.4.10 Detailed Classification Report – Best Model

In [28]:

```
best_preds = all_preds[best_name]
print(f"Classification Report - {best_name}\n")
print(classification_report(y_test, best_preds, target_names=order))
```

A.4.11 Feature Importance – Best Tree-Based Model

In [29]:

```
best_model_map = {
    'Decision Tree': gs_dt.best_estimator_,
    'Random Forest': gs_rf.best_estimator_,
    'XGBoost':       gs_xgb.best_estimator_,
    'LightGBM':      gs_lgb.best_estimator_,
    'k-NN':          None
}
best_model = best_model_map.get(best_name)
if best_model is None:
    best_name_fi = 'LightGBM'
    best_model   = gs_lgb.best_estimator_
else:
    best_name_fi = best_name

fi = pd.Series(best_model.feature_importances_,
               index=feature_cols).sort_values(ascending=False).head
                   (20)

fig, ax = plt.subplots(figsize=(9, 7))
fi.sort_values().plot(kind='barh', ax=ax,
                      color='#1565C0', edgecolor='white')
ax.set_title(f'Top 20 Feature Importances - {best_name_fi}',
             fontsize=12, fontweight='bold')
ax.set_xlabel('Importance Score')
plt.tight_layout()
plt.show()

print("\nTop 5 most important features:")
for i, (feat, score) in enumerate(fi.head(5).items(), 1):
    print(f"  {i}. {feat}: {score:.4f}")
```

A.5 Unsupervised Learning Using Clustering Algorithms

A.5.1 Prepare Data for Clustering

In [30]:

```
X_clust = df_enc[feature_cols].copy()

# Guard: impute any residual NaNs
residual_clust = X_clust.columns[X_clust.isnull().any()].tolist()
if residual_clust:
    print(f"Residual NaNs found in clustering features: {
        residual_clust} "
          "-- applying median imputation.")
    imp_clust = SimpleImputer(strategy='median')
    X_clust   = pd.DataFrame(imp_clust.fit_transform(X_clust),
                             columns=X_clust.columns)
else:
```

```
    print("No residual NaNs in clustering features -- proceeding.")

scaler2 = StandardScaler()
X_sc    = scaler2.fit_transform(X_clust)

assert not np.isnan(X_sc).any(), "X_sc contains NaN -- check pipeline
    ."

# PCA: 50 components captures full variance
pca50   = PCA(n_components=min(50, X_sc.shape[1]), random_state=SEED)
X_pca50 = pca50.fit_transform(X_sc)

# 2-component PCA for visualisation
pca2   = PCA(n_components=2, random_state=SEED)
X_pca2 = pca2.fit_transform(X_sc)

print(f"Variance explained by 50 PCs: "
      f"{pca50.explained_variance_ratio_.sum():.3f}")
print(f"Variance explained by  2 PCs: "
      f"{pca2.explained_variance_ratio_.sum():.3f}")
```

A.5.2 PCA Variance Explained

In [31]:

```
pca_full = PCA(random_state=SEED)
pca_full.fit(X_sc)
cumvar = np.cumsum(pca_full.explained_variance_ratio_)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(range(1, len(cumvar) + 1), cumvar,
        'o-', markersize=3, color='#1565C0')
ax.axhline(0.90, color='red',    linestyle='--', alpha=0.7,
           label='90% threshold')
ax.axhline(0.95, color='orange', linestyle='--', alpha=0.7,
           label='95% threshold')
ax.set_xlabel('Number of Principal Components')
ax.set_ylabel('Cumulative Variance Explained')
ax.set_title('PCA - Cumulative Variance Explained',
             fontsize=12, fontweight='bold')
ax.legend()
ax.set_xlim(1, 55)
plt.tight_layout()
plt.show()

n90 = np.searchsorted(cumvar, 0.90) + 1
n95 = np.searchsorted(cumvar, 0.95) + 1
print(f"Components needed for 90% variance: {n90}")
print(f"Components needed for 95% variance: {n95}")
```

A.5.3 Optimal k Selection – Elbow and Silhouette Methods

In [32]:

```
inertias, sils = [], []
ks = range(2, 9)
```

```
for k in ks:
    km   = KMeans(n_clusters=k, random_state=SEED, n_init=10)
    labs = km.fit_predict(X_pca50)
    inertias.append(km.inertia_)
    sils.append(silhouette_score(X_pca50, labs))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(ks, inertias, 'o-', color='#1565C0', linewidth=2)
ax1.set_title('Elbow Method', fontsize=11, fontweight='bold')
ax1.set_xlabel('Number of Clusters k')
ax1.set_ylabel('Inertia')

ax2.plot(ks, sils, 's-', color='#C62828', linewidth=2)
ax2.set_title('Silhouette Score by k', fontsize=11, fontweight='bold'
   )
ax2.set_xlabel('Number of Clusters k')
ax2.set_ylabel('Silhouette Score')
fig.suptitle('K-Means Cluster Selection Criteria',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

best_k = list(ks)[np.argmax(sils)]
print(f"Best k by silhouette score: {best_k}")
print(f"Silhouette scores: "
      f"{dict(zip(ks, [round(s, 4) for s in sils]))}")
```

A.5.4 K-Means Clustering

In [33]:

```
km_final  = KMeans(n_clusters=best_k, random_state=SEED, n_init=10)
km_labels = km_final.fit_predict(X_pca50)
print(f"K-Means cluster sizes: "
      f"{pd.Series(km_labels).value_counts().sort_index().to_dict()}"
         )
```

A.5.5 Agglomerative (Hierarchical) Clustering

In [34]:

```
hc        = AgglomerativeClustering(n_clusters=best_k, linkage='ward'
   )
hc_labels = hc.fit_predict(X_pca50)
print(f"Agglomerative cluster sizes: "
      f"{pd.Series(hc_labels).value_counts().sort_index().to_dict()}"
         )
```

A.5.6 Hierarchical Dendrogram

In [35]:

```
rng        = np.random.default_rng(SEED)
sample_idx = rng.choice(len(X_pca50), 200, replace=False)
Z          = linkage(X_pca50[sample_idx], method='ward')
```

```
fig, ax = plt.subplots(figsize=(12, 5))
dendrogram(Z, ax=ax, truncate_mode='lastp', p=20,
           leaf_rotation=45, leaf_font_size=8)
ax.set_title('Hierarchical Clustering Dendrogram '
             '(n=200 sample, Ward linkage)',
             fontsize=12, fontweight='bold')
ax.set_xlabel('Sample Index / Cluster Size')
ax.set_ylabel('Euclidean Distance')
plt.tight_layout()
plt.show()
print("The dendrogram shows the natural merge distances. "
      "A clear cut at high distance supports")
print(f"the choice of k={best_k} clusters.")
```

A.5.7 Clustering Evaluation Metrics

In [36]:

```
def cluster_metrics(X, labels, name):
    """Compute Silhouette, Davies-Bouldin, and Calinski-Harabasz."""
    return {
        'Method':                name,
        'Silhouette (up)':       round(silhouette_score(X, labels),
           4),
        'Davies-Bouldin (down)': round(davies_bouldin_score(X, labels
           ), 4),
        'Calinski-Harabasz (up)':round(calinski_harabasz_score(X,
           labels), 2)
    }

clust_eval = pd.DataFrame([
    cluster_metrics(X_pca50, km_labels, 'K-Means'),
    cluster_metrics(X_pca50, hc_labels, 'Agglomerative (Ward)')
]).set_index('Method')
print(clust_eval)
print("\n(up) = higher is better    (down) = lower is better")
```

A.5.8 PCA Visualisation of Cluster Assignments

In [37]:

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, labs, title in zip(axes,
                           [km_labels, hc_labels],
                           ['K-Means', 'Agglomerative (Ward)']):
    sc = ax.scatter(X_pca2[:, 0], X_pca2[:, 1], c=labs,
                    cmap='Set1', alpha=0.55, s=12, linewidths=0)
    ax.set_title(f'{title} - PCA Projection (k={best_k})',
                 fontsize=11, fontweight='bold')
    ax.set_xlabel('PC1 ({:.1f}%)'.format(
        pca2.explained_variance_ratio_[0] * 100))
    ax.set_ylabel('PC2 ({:.1f}%)'.format(
        pca2.explained_variance_ratio_[1] * 100))
    plt.colorbar(sc, ax=ax, label='Cluster')
fig.suptitle('Clustering Results - PCA 2D Projection',
             fontsize=13, fontweight='bold')
```

```
plt.tight_layout()
plt.show()
```

A.5.9 t-SNE Visualisation

In [38]:

```
# t-SNE on a 500-sample subset for computational tractability
tsne   = TSNE(n_components=2, random_state=SEED,
              perplexity=40, max_iter=300)
X_tsne = tsne.fit_transform(X_sc[:500])

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].scatter(X_tsne[:, 0], X_tsne[:, 1], c=km_labels[:500],
                cmap='Set1', alpha=0.6, s=15, linewidths=0)
axes[0].set_title('t-SNE - K-Means Cluster Assignment',
                  fontsize=11, fontweight='bold')
axes[0].set_xlabel('t-SNE 1')
axes[0].set_ylabel('t-SNE 2')

true_sub = y.values[:500]
sc2 = axes[1].scatter(X_tsne[:, 0], X_tsne[:, 1], c=true_sub,
                      cmap='RdYlGn_r', alpha=0.6, s=15, linewidths=0)
axes[1].set_title('t-SNE - True Escalation Level',
                  fontsize=11, fontweight='bold')
axes[1].set_xlabel('t-SNE 1')
axes[1].set_ylabel('t-SNE 2')
plt.colorbar(sc2, ax=axes[1], ticks=[0, 1, 2, 3],
             label='0=No Esc  1=Minor  2=Serious  3=Legal')
fig.suptitle('t-SNE Visualisation (n=500 sample)',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

A.5.10 Cluster Composition by Escalation Level

In [39]:

```
ct          = pd.crosstab(km_labels, y.values, normalize='index') *
   100
ct.columns  = [order[i] for i in ct.columns]
ct.index    = [f'Cluster {i}' for i in ct.index]

fig, ax = plt.subplots(figsize=(8, 4))
sns.heatmap(ct, annot=True, fmt='.1f', cmap='YlOrRd',
            linewidths=0.5, ax=ax, annot_kws={'size': 11})
ax.set_title('Cluster Composition by Escalation Level (%)',
             fontsize=12, fontweight='bold')
ax.set_xlabel('Escalation Level')
ax.set_ylabel('Cluster')
plt.tight_layout()
plt.show()

print("\nInterpretation:")
for i, row in ct.iterrows():
    dominant = row.idxmax()
```

```
    print(f"  {i}: Dominated by '{dominant}' ({row[dominant]:.1f}%)")
```

A.5.11 Cluster Operational Profiles

In [40]:

```
df_profiled            = df.copy()
df_profiled['cluster'] = km_labels
profile_num = ['wait_time_seconds', 'SLA_breaches_last_6m',
               'number_of_transfers', 'sentiment_score',
               'resolution_time_minutes', 'monthly_fee',
               'behaviour_mean']
profile_df = df_profiled.groupby('cluster')[profile_num].mean()
print("Cluster means (raw):")
print(profile_df.round(3))

profile_norm = ((profile_df - profile_df.min()) /
                (profile_df.max() - profile_df.min() + 1e-9))

fig, ax = plt.subplots(figsize=(10, 5))
profile_norm.T.plot(kind='bar', ax=ax,
                    colormap='Set1', edgecolor='white')
ax.set_title('Normalised Cluster Profiles - Key Operational Metrics',
             fontsize=12, fontweight='bold')
ax.set_ylabel('Normalised Mean Value (0-1)')
ax.tick_params(axis='x', rotation=35)
ax.legend(title='Cluster', fontsize=9)
plt.tight_layout()
plt.show()
```

References

Bentéjac, C., Csörgő, A., and Martínez-Muñoz, G. (2020). A comparative analysis of gradient boosting algorithms. *Artificial Intelligence Review*, 54(3):1937–1967.

Emmanuel, T., Maupong, T., Mpoeleng, D., Semong, T., Banyatsang, M., and Tabona, O. (2021). A survey on missing data in machine learning. Preprint.

Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35, pages 507–520.

Ikotun, A. M., Ezugwu, A. E., Abualigah, L., Abuhaija, B., and Heming, J. (2023). K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data. *Information Sciences*, 622:178–210.

Kapoor, S. and Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804.

Raschka, S., Patterson, J., and Nolet, C. (2020). Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence. *Information*, 11(4):193.

Thabtah, F., Hammoud, S., Kamalov, F., and Gonsalves, A. (2020). Data imbalance in classification: Experimental evaluation. *Information Sciences*, 513:429–441.